

SynchronAlse

Execution Roadmap — Use Case 01: Automated Visual Auditing

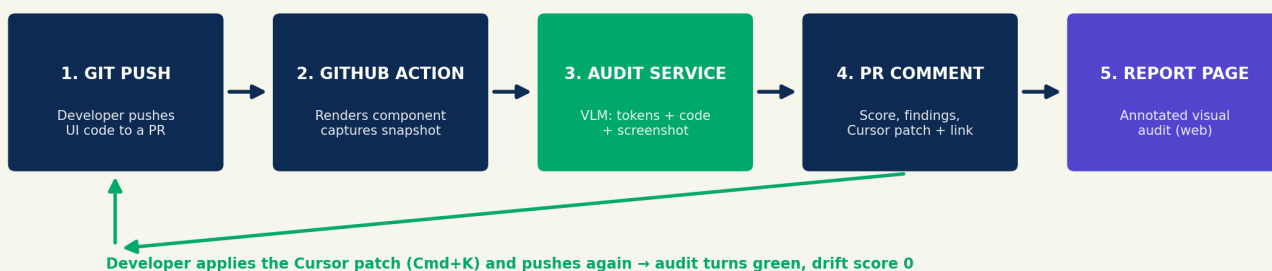
RAISE Hackathon 2026 · Cursor Track · Submission deadline: Sunday, July 5th, 12:00 PM

1. What we are building

A **CI/CD-native design auditor** with a **visual audit report page** as its demo surface. The product lives inside the Git workflow: on every push, a GitHub Action renders the changed component, the AI engine classifies every deviation, and the developer receives instant feedback — a PR comment with a Cursor-compatible patch, plus a link to a visual report.

END-TO-END PIPELINE

The product lives inside the Git workflow — the AI engine (step 3) is the star



The full loop: push → snapshot → classification → feedback → fix in Cursor → green.

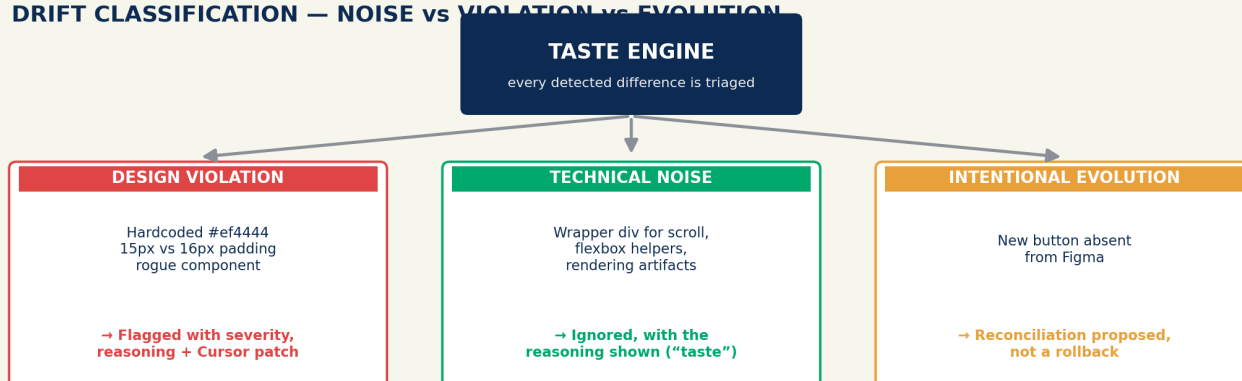
Strategic order — non-negotiable: the pipeline (steps 1–4) is the product and is built on Saturday. The report page (step 5) is the demo amplifier, built only once the pipeline works. If it isn't demo-solid by Sunday 10:00 AM, it gets cut and the PR comment carries the demo alone.

Frozen scope. No PR list, navigation, auth, settings or multi-repo in the frontend — ONE page per audit. No QA Lens, no bidirectional sync, no dynamic Figma scraper, no CLI. Those are roadmap items, mentioned verbally at the end of the pitch.

The Taste Engine — what makes us different

Every detected difference is triaged into three classes. Getting the middle one right — ignoring technical noise *with visible reasoning* — is the judged “taste” and the core of the project.

DRIFT CLASSIFICATION — NOISE vs VIOLATION vs EVOLUTION



2. Team & responsibilities

Role	Owner	Main responsibility	Deliverable
R1 — AI Lead	AI PhD	Classification prompt: Noise vs Violation vs Evolution	Final prompt + validated JSON schema + measured accuracy
R2 — Data	Data expert	Golden dataset: tokens, hero component, drift PRs, bbox annotations	Hero component + 6 drift cases as real open PRs + ground truth
R3 — Backend	Big Data 1	Audit service: VLM orchestration, result storage, report data	/audit endpoint + stored results + GET /report/{id}
R4 — CI/CD	Big Data 2	The GitHub Action — critical path from hour one	Push → render + snapshot → call audit → post PR comment
R5 — Frontend	Platform eng	PR comment template (Sat) → Audit Report page (Sun) + video + submission	Polished comment, one-screen report page, 1-min video, form submitted

Golden rule: every pair sharing an interface agrees on the JSON contract **before** writing code (§4). R3 mocks it in hour one so R4 and R5 never wait on R1.

3. Hero component & golden dataset (R2, top priority)

Component: an interactive StatusCard (icon + title + description + action button, with default / warning / danger variants). Rich enough for structural and token drift, fast to render in CI.

Hardcoded design system (~20 tokens): color-primary, color-danger, color-warning, color-surface, color-text · spacing-sm 8px, spacing-md 16px, spacing-lg 24px · font-heading, font-body · radius-md 8px...

#	PR / Case	Expected class	Why it's in the dataset
1	Hardcoded #ef4444 instead of var(--color-danger)	Design Violation	Obvious case, opens the demo
2	15px padding instead of 16px (spacing-md)	Design Violation	Pixel-level precision
3	Wrapper div added for scroll/flexbox handling	Technical Noise (ignored)	THE “taste” moment — a naive linter would flag it
4	Component rebuilt from scratch mimicking the StatusCard	Design Violation (rogue)	Subgraph matching
5	New secondary button absent from Figma	Intentional Evolution	Nuance: propose reconciliation, don't roll back
6	Cases 1+3 combined in the same file	Mixed	Proves the system separates signals

For each PR: drifted React code, CI-rendered screenshot, and **ground truth** (expected classification + precise violation + **bounding-box coordinates** of the drift zone, needed for the annotated screenshot). Real open PRs mean the demo repo tells the story by itself when judges browse it.

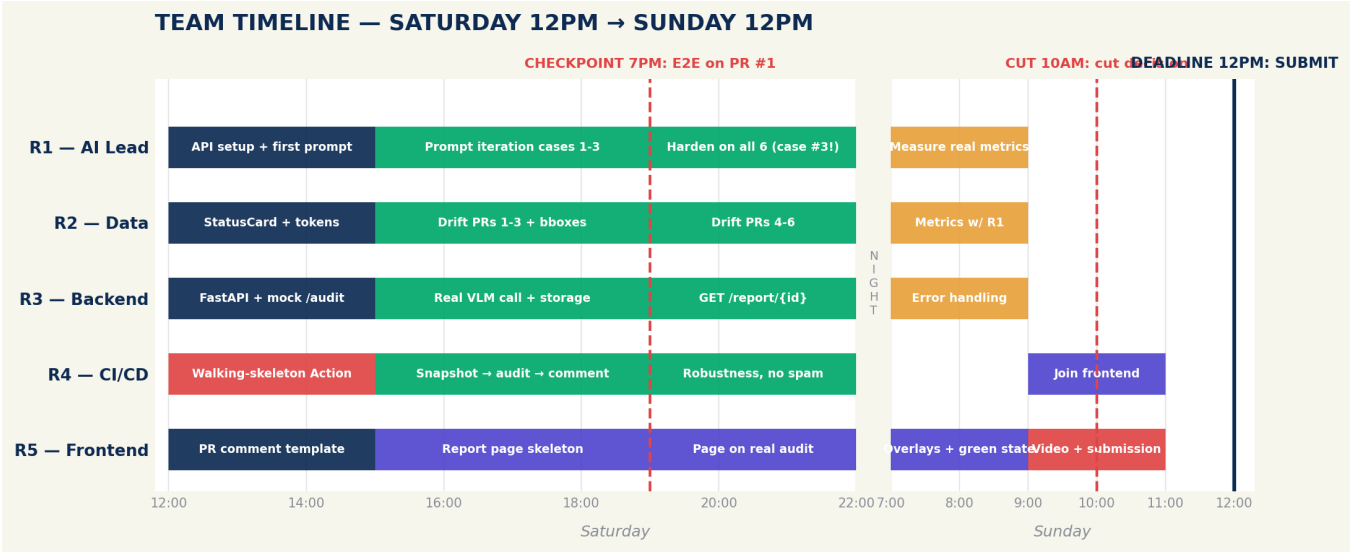
4. The JSON contract (freeze at H+1, everyone, 20 min max)

One payload drives everything: the PR comment, the report page, and the metrics script.

```
{
  "audit_id": "pr-6-run-3", "pr_number": 6, "drift_score": 72,
  "screenshot_url": "/artifacts/pr-6-run-3.png",
  "findings": [{
    "type": "token_violation",
    "classification": "design_violation",
    "severity": "high",
    "location": "StatusCard.tsx:14",
    "bbox": [120, 48, 340, 92],
    "expected": "var(--color-danger)", "actual": "#ef4444",
    "reasoning": "Hex value matches token color-danger exactly; hardcoding breaks theme propagation.",
    "cursor_patch": {
      "prompt": "Replace the hardcoded #ef4444 on line 14 with var(--color-danger) from our design tokens.",
      "diff": "- color: #ef4444; + color: var(--color-danger);" }
  ]},
  "ignored_as_noise": [{
    "element": "div.scroll-wrapper",
    "reasoning": "Structural wrapper required for overflow handling; no visual or semantic impact." }],
  "evolution_proposals": []
}
```

- **bbox** powers the highlighted drift zones on the report page (LLM returns approximate regions; R2's ground truth validates them).
- **cursor_patch** carries a paste-ready Cursor prompt *and* a suggested diff (rendered as a GitHub suggestion block).
- The backend **stores every audit** by audit_id; the report page is a pure renderer of this payload.
- Once frozen: R3 serves it hardcoded, R4 builds the Action against the mock, R5 designs comment + page against the mock, R1 makes the real LLM produce this format.

5. Timeline — Saturday 12PM → Sunday 12PM



Each role works in parallel against the mocked contract; red markers are the three hard checkpoints.

Slot	Focus	Key actions
Sat now → 3PM	Foundations	20-min kickoff: freeze JSON contract, create tool + demo repos. R4 ships a walking-skeleton Action (hardcoded comment) to prove trigger + permissions in hour one. R3 deploys mocked /audit. R5 designs the comment template. R1 sets up API access. R2 builds StatusCard + tokens.
Sat 3 → 7PM	Core loop	R4: Playwright render in CI → snapshot → /audit → formatted comment. R2 delivers PRs 1-3. R1 iterates the prompt until stable (3 identical runs). R3 wires the real VLM + storage. R5: comment on real payloads + report page skeleton.
CHECKPOINT 7PM		Pushing to PR #1 produces a real AI-generated comment, end to end. If not, everyone converges on the blocker. The report page does NOT gate this checkpoint.

Slot	Focus	Key actions
Sat 7 → 10PM	Extend	R2 delivers PRs 4-6. R1 hardens across all 6 — case #3 is won or lost here. R4: audit only changed files, update comment on re-push (no spam). R5: report page renders a real stored audit. Before leaving: push everything + written list of known bugs.
Sun 7 → 9AM	Hardening	E2E test of all 6 PRs, twice each; push-to-comment latency target < 60s. R1+R2 measure REAL metrics (the only numbers used in the pitch). R3: timeouts, retries, soft-fail comment. R5 (+R4): drift-zone overlays + green "resolved" state.
CUT 10AM		If the report page isn't demo-solid, freeze it out — the PR comment carries the demo. No debate, no sunk-cost.
Sun 10 → 11AM	Demo & video	Rehearse the flow 3×: push → comment → report → patch in Cursor → push → green. R5 records the 1-min video of exactly that loop (YouTube/Loom). Plan B: full screen capture + one PR left pre-commented.
Sun 11 → 12PM	Submission	R5 submits the form (video, description, repos) — do NOT wait until 11:55. Everyone else: repo cleanup, no API keys in git history, .env.example, license, README "built during the event" section.
12:15 → judging	Pitch	3 minutes, two speakers max: 20s problem → 30s architecture → 90s live demo → 20s real metrics → 20s roadmap (QA Lens, bidirectional sync).

6. Demo script — 3 minutes, built as a crescendo

3-MINUTE DEMO — BUILT AS A CRESCENDO

50% of the score is the demo — no slides, only the working product



1. **Opening (PR #1):** push a commit live. “Twenty seconds later, SynchronAlse has reviewed the design.” → the comment appears: violation, reasoning, patch, report link.
2. **The visual (report page):** annotated screenshot, drift zone highlighted, expected vs actual side by side. Three seconds of silence — let it land.
3. **The taste moment (PR #3):** “A classic linter would have flagged this wrapper div. Ours didn't — here's its reasoning.” → the *Ignored as technical noise* section.
4. **The Cursor climax (PR #6):** copy the patch, Cmd+K in Cursor, code fixed, push → comment updates, report turns green, **drift score 0**.
5. **Real numbers:** “On our test set: X/6 correct, Y noise cases correctly ignored.”

7. Risks & mitigations

Risk	Likelihood	Mitigation
LLM misclassifies case #3 (noise vs violation)	High	R1's main job; few-shot examples; fallback = whitelist of known structural patterns
Playwright-in-CI flaky	High — critical path	Pin browser versions; pre-built Docker image; fallback = pre-rendered screenshots committed per PR
Report page eats the schedule	Medium	Hard 10AM cut decision; page is a pure renderer of stored JSON, zero business logic
Push-to-comment latency too long live	Medium	Compressed images, warm backend; one pre-commented PR as backup; recorded video
Non-JSON LLM output	Medium	response_format JSON if available, otherwise retry with the error in context
GitHub token permissions / rate limits	Medium	Walking skeleton proves permissions in hour one; dedicated bot token
Bboxes from the LLM are imprecise	Medium	Generous rounded highlights, not pixel-perfect boxes; ground-truth bboxes as demo fallback
API quota exhausted	Low	Two providers configured Saturday (Gemini + one fallback)
Running out of time	Certain :)	Sacrifice order: report page → PR #6 → PR #5 → comment-update-on-repush

8. Definition of Done — submission checklist

- Public tool repo + public demo repo, README with architecture + run instructions + “built during the event” section
- All 6 drift cases live as open PRs in the demo repo, each with its AI-generated comment visible
- GitHub Action: push → snapshot → classification → PR comment, unassisted
- Report page renders at least the 6 demo audits (or documented cut, comment-only fallback)
- cursor_patch verified: applying it in Cursor and pushing turns the audit green

-
- Real metrics measured and written in the README
 - 1-min video uploaded + form submitted before 12:00 PM
 - No API keys anywhere in git history
 - Pitch rehearsed 3 times, under 3 minutes
-

SynchronAlse — pipeline first, report page second, taste above all. Good luck at the 7PM checkpoint.